C# Naming Conventions

Quick reference for developers

A standard naming convention followed across a C# / ASP.NET codebase. Consistent naming makes code easier to read, review, and maintain — especially across multiple modules and contributors.

Item	Convention	Example
Class	PascalCase	`EmployeeController, EmployeeVM`
Method	PascalCase	`Create(), GetEmployee()`
Property	PascalCase	`EmployeeId, FirstName`
Interface	Prefix I	`IEmployeeService`
Enum	PascalCase	`EmployeeStatus`
Enum Values	PascalCase	`Active, Inactive`
Local Variable	camelCase	`employee, user, roleName`
Parameter	camelCase	`id, employeeVM`
Private Field	_camelCase	`_context, _logger`
Constant	PascalCase	`MaxFileSize`
Async Method	Suffix Async	`CreateAsync(), LoadEmployeeAsync()`

Notes

• Interfaces are always prefixed with **I** so their purpose is recognizable at a glance, even without an IDE (e.g. `IEmployeeService`, not `EmployeeServiceInterface`).

• Private fields use a leading underscore (`_camelCase`) to visually distinguish them from parameters and local variables at a glance inside a method body.

• Any method that performs I/O-bound or awaitable work (database calls, HTTP requests, file access) must be suffixed with **Async** and should return `Task` or `Task<T>`.

• Acronyms longer than two letters follow PascalCase capitalization rules (e.g. `HtmlParser`, not `HTMLParser`); two-letter acronyms stay fully capitalized (e.g. `IOException`).